

# Sertifikalar ve Kriptografik Yöntemler

CEN429 Güvenli Programlama — Hafta 10

Dr. Öğr. Üyesi Uğur CORUH · 20.11.2026

## Bugünün planı (3 saat)

---

| Saat | Bölüm | Konu                                                                 |
|------|-------|----------------------------------------------------------------------|
| 1    | 0–3   | Temel kavramlar · algoritma/anahtar seçimi · kipler/dolgu · MAC/HMAC |
| 2    | 4–7   | RSA/ECC · OAEP/PSS · dijital imza · Diffie–Hellman · PKI             |
| 3    | 8–13  | X.509 · zincir · CRL/OCSP · HSM/PKCS#11 · kuantum sonrası · proje    |

## Kısa tarihçe — anahtarlar, sertifikalar, PKI

- **1976–77** — Diffie–Hellman ve **RSA**: tanımadığınla güvenli konuşma
- **1988** — **X.509** sertifika biçimi; kimliği bir **CA imzası** taşır
- **1995** — ticari CA'lar ve **PKI**; ardından **CRL** ve **OCSP** (iptal)
- **2014** Heartbleed · **2015** Let's Encrypt · **2018** TLS 1.3 · **2022–24 PQC** (Kyber/Dilithium)

Bugünkü kurallar (doğru kip/dolgu, zincir doğrulama, iptal) bu acı derslerden çıktı.

## Bu hafta nereye oturuyor?

---

- **3. hafta:** kriptoya giriş (gizlilik, bütünlük).
- **Bu hafta (10):** doğru **algoritma/kip/dolgu seçimi**, anahtar yaşam döngüsü, **PKI**.
- **11. hafta:** anahtar yazılımda korunuyorsa → whitebox.

# Öğrenme çıktısı

---

Bu hafta **ÖÇ.2 / ÖÇ.4** üstünedir.

Sonunda yapabileceğiniz:

- Doğru algoritma, kip, dolgu ve anahtar uzunluğu seçmek
- İmza ve anahtar değişiminin **tuzaklarını** görmek
- Bir sertifika **zincirini** doğru doğrulamak

# Ana fikir

---

Kriptoda hatalar genelde algoritmada değil, **kullanımdadır**: yanlış kip, yanlış dolgu, doğrulanmayan imza, denetlenmeyen zincir.

Bugün doğru **kullanımı** öğreneceğiz.

# 0. Temel kavramlar (sıfırdan)

## Simetrik vs asimetrik (hatırlatma)

---

- **Simetrik:** tek anahtar, şifrele ve çöz (AES). Hızlı.
- **Asimetrik:** açık + gizli anahtar çifti (RSA, ECC). Yavaş ama anahtar dağıtımı kolay.
- Pratikte **birlikte:** asimetrik ile anahtar taşı, simetrik ile veri şifrele.

# Blok şifre ve kip

---

- **Blok şifre:** sabit boyutlu bloğu şifreler (AES: 16 bayt).
- **Kip (mode):** blokları **nasıl** zincirleyeceğimiz (CBC, GCM...).
- Kip seçimi güvenliğin **kalbidir**.

# Dolgu (padding)

---

- **Dolgu:** veri blok boyutunun katı değilse tamamlama.
- Bazı kiplerde gerekir (CBC), bazılarında **yok** (GCM).
- Yanlış dolgu işleme → **dolgu kâhini** saldırısı.

# AEAD nedir?

---

- **AEAD (Authenticated Encryption with Associated Data):** gizlilik + bütünlüğü **birlikte** veren şifreleme.
- Örnek: AES-GCM, ChaCha20-Poly1305.
- Modern tercih: ayrı MAC uğraşma, AEAD kullan.

# MAC ve HMAC

---

- **MAC (Message Authentication Code):** bir mesajın **değişmediğini** ve doğru taraftan geldiğini kanıtlayan etiket (simetrik).
- **HMAC:** özet fonksiyonuna dayalı yaygın bir MAC.
- Bütünlük için.

# Encrypt-then-MAC — şema

## Encrypt-then-MAC: doğru sıra

alıcı geçersiz metni hiç çözmez → dolgu kâhini kapanır



Yanlış sıra (önce çöz, sonra doğrula) dolgu hatası sızdırır ve oracle açar.

## Özet (hash)

---

- **Özet:** veriden hesaplanan sabit parmak izi (SHA-256).
- Tek yönlü: özetten veri geri gelmez.
- İmza ve MAC'in yapı taşı.

# Dijital imza

---

- **İmza**: asimetrik; **gizli** anahtarla imzala, **açık** anahtarla doğrula.
- Sağlar: bütünlük + **inkâr edilemezlik** (kim imzaladı).
- MAC'ten farkı: asimetrik, herkes doğrulayabilir.

## RSA ve eliptik eğri (ECC)

---

- **RSA**: klasik asimetrik; büyük anahtarlar (2048+ bit).
- **ECC**: eliptik eğri; aynı güvenlik **daha küçük** anahtarla (Ed25519, X25519).
- Modern tercih giderek ECC.

# Diffie–Hellman (DH)

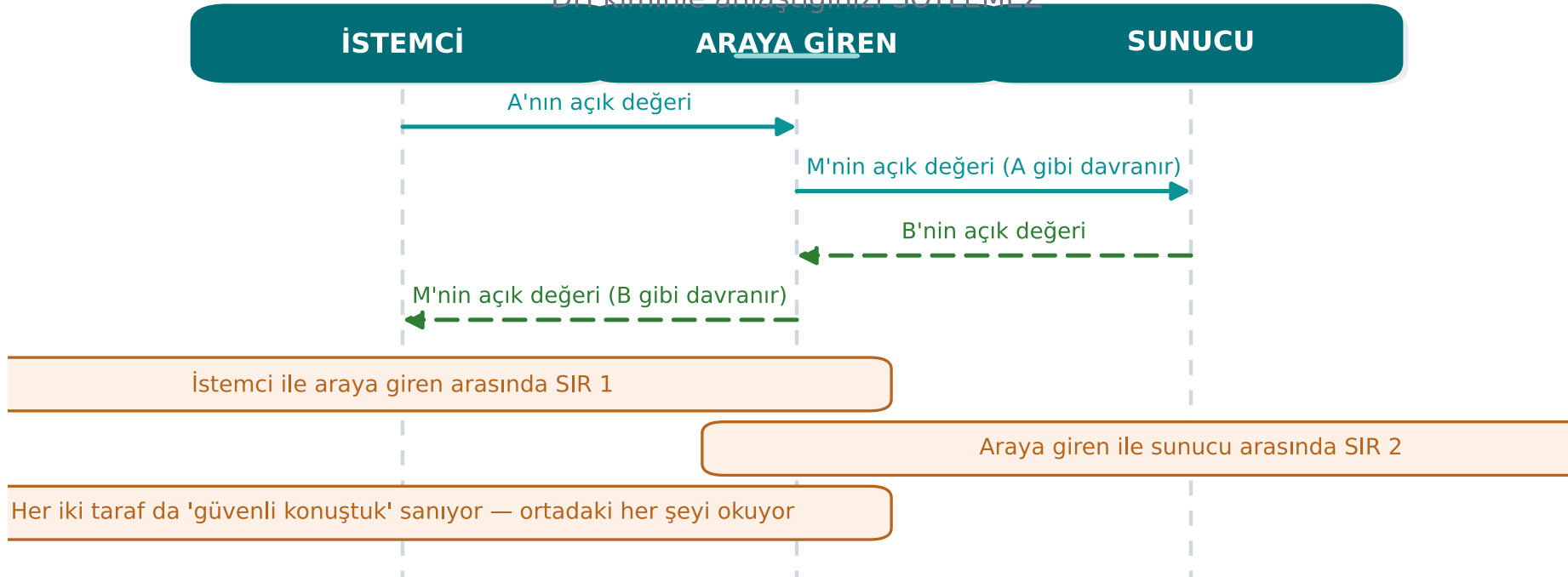
---

- **DH:** iki tarafın, gizli anahtar **paylaşmadan** ortak bir sır türetmesi.
- Ağ üzerinden anahtar anlaşması.
- Kimlik doğrulanmazsa **araya girme** (MITM) riski.

# Kimliksiz DH ve MITM — şema

## Kimliksiz Diffie-Hellman: araya girme

DH kiminle anlaştığınızı SÖYLEMEZ



Çözüm: DH'yi kimlik kanıtıyla bağla (sertifika / imzalı DH) — TLS 1.3 bunu yapar.

# PKI ve CA

---

- **PKI (Public Key Infrastructure):** "hangi açık anahtar kime ait?" sorusunu çözen güven sistemi.
- **CA (Certificate Authority):** sertifikaları imzalayan güvenilir taraf.
- Kök CA → ara CA → sunucu sertifikası.

## Sertifika ve X.509

---

- **Sertifika:** bir açık anahtarı bir kimliğe (alan adı) bağlayan, CA'nın imzaladığı belge.
- **X.509:** sertifikaların standart biçimi.
- İçinde: konu, açık anahtar, geçerlilik, imza, SAN.

## CRL ve OCSP

---

- **CRL (Certificate Revocation List):** iptal edilmiş sertifikaların **listesi**.
- **OCSP:** bir sertifikanın iptal durumunu **anlık** sorma.
- "Bu sertifika hâlâ geçerli mi?" sorusu.

# HSM, PKCS#11, SoftHSM

---

- **HSM:** anahtarları saklayan/işleten özel **donanım**; anahtar dışarı çıkmaz.
- **PKCS#11:** anahtar modülleriyle konuşmanın standart arayüzü.
- **SoftHSM:** HSM'in yazılım benzetimi (test için).

## Kuantum sonrası (PQC)

---

- **PQC (Post-Quantum Cryptography):** kuantum bilgisayara dayanıklı algoritmalar.
- Bugünkü RSA/ECC gelecekte tehdit altında.
- Standartlaşma sürüyor (ör. ML-KEM).

# Şimdi hazırız

---

Terimler:

simetrik/asimetrik · blok şifre/kip · dolgu · AEAD · MAC/HMAC · özet · imza · RSA/ECC · DH · PKI/CA ·  
sertifika/X.509 · CRL/OCSP · HSM/PKCS#11 · PQC

Şimdi: doğru algoritma ve anahtar seçimi.

# 1. Algoritma ve anahtar seçimi

# Güncel, standart, iyi kullanılan

---

Üç kural:

- **Güncel:** kırılmamış algoritma (AES, SHA-256, Ed25519).
- **Standart:** kendi kriptonu yazma; denenmiş kütüphane.
- **İyi kullanılan:** doğru kip, dolgu, anahtar yönetimi.

## Kaçınılacaklar

---

- MD5, SHA-1 (özet için), DES/3DES, RC4.
- ECB kipi.
- Kendi "şifreleme" algoritman.
- Sabit/öngörülebilir IV veya anahtar.

# Anahtar uzunluğu

---

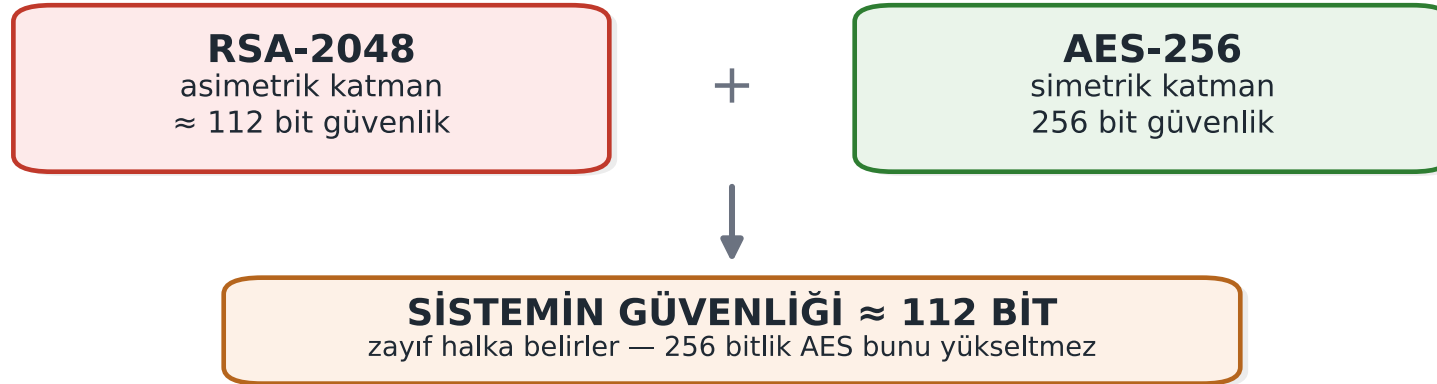
| Amaç     | Öneri (yaklaşık)              |
|----------|-------------------------------|
| Simetrik | AES-128 (yeter), AES-256      |
| RSA      | $\geq 2048$ , tercih 3072     |
| ECC      | 256-bit ( $\approx$ RSA-3072) |
| Özet     | SHA-256+                      |

---

# En zayıf halka kuralı

## En zayıf halka kuralı

sistemin gücü, en güçlü parçası kadar değildir



Katmanları ayrı ayrı değil, en düşük güvenlik seviyesine göre değerlendirin.

- Bileşenleri **dengeli** seç.
- 128 bit hedefliyorsan RSA-3072 ya da ECC.

## 2. Blok şifre kipleri ve dolgu

# OAEP / PSS — şema

## RSA dolgusu: hangisi nerede?

dolgu, algoritmanın güvenliğinin ayrılmaz parçasıdır

### OAEP

RSA ile ŞİFRELEME  
rastgeleleştirilmiş dolgu

ham RSA asla kullanılmaz

### PSS

RSA ile İMZA  
rastgeleleştirilmiş dolgu

PKCS#1 v1.5 yerine tercih

Eliptik eğride karşılıkları: X25519 (anahtar değişimi), Ed25519 (imza).

## ECB · asla

---

- **ECB:** her bloğu bağımsız şifreler.
- Aynı blok → aynı şifreli çıktı → **desen sızar.**
- Ünlü "ECB penguen" örneği.

ECB kullanma.

# ECB neden kötü — şema

## ECB neden asla kullanılmaz?

ünlü 'ECB pengueni' bu yüzden görünür kalır

### ECB

her blok BAĞIMSIZ şifrelenir

aynı düz metin bloğu →  
aynı şifreli blok  
→ DESENLER görünür

### CBC / CTR / GCM

her blok öncekine ya da  
nonce'a bağlı

aynı metin → farklı şifreli  
→ desen sızmaz

Kip seçimi şifrelemenin kendisi kadar önemlidir; varsayılan tercih AEAD (GCM).

## CBC · dikkatli

---

- **CBC**: her blok bir öncekiyle zincirlenir; **IV** gerekir.
- IV **rastgele** ve tekrarsız olmalı.
- Tek başına **bütünlük vermez** → ayrı MAC şart.

# CBC + dolgu = risk

---

- CBC dolgu gerektirir.
- Yanlış dolgu işleme → **dolgu kâhini** (birazdan).
- Bu yüzden modern tercih **AEAD**.

## GCM · modern tercih (AEAD)

---

- **AES-GCM:** gizlilik + bütünlük birlikte.
- Dolgu **yok**; bir **nonce** (tek kullanımlık sayı) gerekir.
- Nonce **asla** tekrar etmemeli (aynı anahtar altında).

## Nonce/IV kuralı

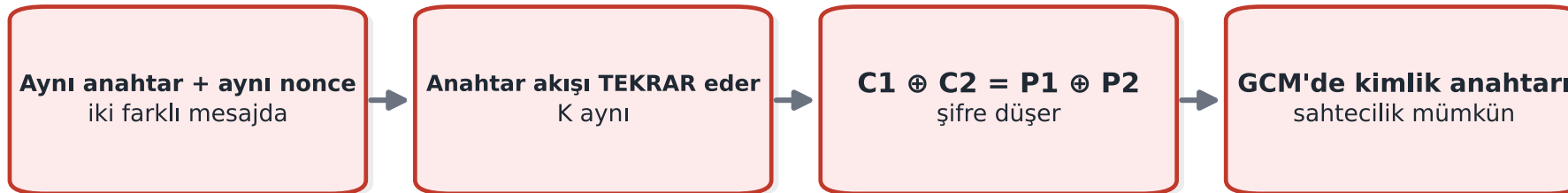
---

- IV/nonce **benzersiz** olmalı.
- GCM'de nonce tekrarı **felakettir** (anahtar/veri sızabilir).
- Sayaç ya da rastgele (yeterli uzunlukta) kullan.

# Nonce tekrarı — şema

## Nonce tekrarı neden felaket?

gizlilik VE bütünlük birlikte çöker



Nonce gizli olmak zorunda değil ama anahtar başına BENZERSİZ olmak zorundadır.

# ChaCha20-Poly1305

---

- AES-GCM'e AEAD alternatifi.
- Donanım AES desteği yoksa hızlı.
- Aynı nonce kuralı geçerli.

# Dolgu kâhini (padding oracle)

## Sorun · adım adım

---

1. Sunucu, CBC çözerken **dolgu geçersiz** ve **MAC geçersiz** için **farklı** yanıt/zaman verir.
2. Saldırgan şifreli metni değiştirip yanıtlara bakar.
3. Yanıt farkından **düz metni** bayt bayt çözer.

## Neden oluyor?

---

- Hata mesajı/zamanı, iç durumu **sızdırıyor**.
- Saldırgan bunu bir **kâhin** (oracle) gibi kullanıyor.

## Çözüm

---

- **AEAD kullan** (GCM): dolgu yok, tek doğrulama.
- CBC zorundaysan: **encrypt-then-MAC** + **tek ve aynı** hata.
- Zamanlama farkı da sızıntıdır → sabit zamanlı.

# Ders

---

Şifreleme tek başına yetmez; **bütünlük** ve **tutarlı hata** şart.

Dolgu kâhini, "kullanım hatası"nın klasik örneğidir.

## Bölüm 1–2 — kısa sinama

---

1. En zayıf halka kuralını bir örnekle açıklayın.
2. ECB neden kullanılmaz?
3. GCM'de nonce tekrarı neden felakettir?
4. Dolgu kâhinini AEAD nasıl kapatır?

## Bölüm 1–2 — cevaplar

---

1. Sistem **en zayıf bileşeni** kadar güvenli. Örn. AES-256 kullanıp anahtarı düz dosyada tutmak → anahtar en zayıf halka.
2. **ECB** aynı açık bloğu aynı şifreli bloğa çevirir → desen sızar, anlamsal güvenlik yok.
3. Aynı anahtar+nonce **anahtar akışını** tekrarlar (gizlilik çöker) ve GHASH **kimlik anahtarı** ele geçirilir → felaket.
4. **AEAD** önce etiketi doğrular, yanlış etiketli metni **çözmeden reddeder** → dolgu hatası için **oracle** kalmaz.

## 3. MAC, HMAC ve bütünlük

# HMAC neden — şema

## Neden HMAC, neden $H(K \parallel m)$ değil?

kendi MAC'inizi kurmayın

### $H(K \parallel m)$

Merkle-Damgård özetlerde  
UZUNLUK UZATMA saldırısı

saldırgan K'yı bilmeden  
 $H(K \parallel m \parallel ek)$  üretebilir

### HMAC

iç içe iki özetleme  
ipad / opad ile

uzunluk uzatma KAPALI  
standart ve kanıtlı

Doğrulamayı sabit zamanlı karşılaştırmayla yapın (CRYPTO\_memcmp).

## Neden MAC?

---

- Şifreleme gizliliği verir, **bütünlüğü vermez**.
- Saldırgan şifreli metni değiştirebilir.
- MAC: mesaj **değişmedi** ve doğru taraftan geldi.

# HMAC

---

- Özet fonksiyonuna dayalı MAC ( HMAC-SHA256 ).
- Simetrik anahtar; iki taraf aynı anahtarı bilir.
- Doğrulama **sabit zamanlı** karşılaştırmayla.

## Doğru sıra · encrypt-then-MAC

```
1. şifrele:  c = ENC(k1, m)
2. MAC'le:  t = MAC(k2, c)
3. gönder:  c || t
```

- Önce şifrele, **sonra** şifreli metni MAC'le.
- MAC geçmezse **çözmeye bile kalkma**.

## Yanlış sıralar

---

- **MAC-then-encrypt**: dolgu kâhinine açık.
- **Encrypt-and-MAC**: MAC düz metni sızdırabilir.
- Doğrusu: **encrypt-then-MAC** (ya da AEAD, zaten böyle yapar).

## Yeniden oynatma (replay)

---

- Saldırgan geçerli bir mesajı **tekrar** gönderir.
- MAC geçerli (mesaj değişmedi) ama işlem **tekrarlanır**.
- Çözüm: **nonce**, zaman damgası, sayaç.

## AEAD hepsini yapar

---

- AES-GCM: şifreleme + bütünlük **birlikte**.
- "İlişkili veri" (AAD) ile başlık/bağlamı da doğrular.
- Modern tercih: ayrı MAC yerine AEAD.

## 4. Asimetrik: RSA ve ECC

# Simetrik ↔ asimetrik — şema

## Simetrik ve asimetrik: hangisi ne için?

pratikte HİBRİT: asimetrikle anahtar kur, simetrikle veri şifrele

### SİMETRİK (AES)

tek anahtar  
ÇOK HIZLI

sorun: anahtarı nasıl paylaşırsın?

### ASİMETRİK (RSA/EC)

açık + gizli anahtar çifti  
YAVAŞ

anahtar dağıtımı ve İMZA çözülür

TLS tam olarak bunu yapar: ECDHE ile anahtar, AES-GCM ile veri.

## RSA · iki iş

---

- **Şifreleme:** açık anahtarla şifrele, gizli anahtarla çöz.
- **İmza:** gizli anahtarla imzala, açık anahtarla doğrula.
- Büyük anahtarlar (2048+).

## RSA · dolgu şart

---

- Ham RSA **güvensizdir**.
- **Şifreleme için**: OAEP dolgusu.
- **İmza için**: PSS dolgusu.
- Eski PKCS#1 v1.5: mümkünse kaçın.

## OAEP vs PSS

---

- **OAEP:** RSA **şifreleme** dolgusu.
- **PSS:** RSA **imza** dolgusu.
- Karıştırma: OAEP imza için değil, PSS şifreleme için değil.

## ECC · neden?

---

- Aynı güvenlik, **daha küçük** anahtar (256-bit  $\approx$  RSA-3072).
- Daha hızlı, daha az yer.
- Mobil/gömülü için ideal.

## Ed25519 ve X25519

---

- **Ed25519**: modern **imza** algoritması.
- **X25519**: modern **anahtar anlaşması** (DH).
- Karıştırma: Ed25519 imza, X25519 anahtar.

## Simetrik + asimetrik birlikte

1. X25519 ile ortak sır türet
2. Ondan bir AES anahtarı çıkar (KDF)
3. AES-GCM ile veriyi şifrele

- Asimetrik: anahtar taşı. Simetrik: veri şifrele.
- TLS bunu yapar.

## Bölüm 3–4 — kısa sinama

---

1. Encrypt-then-MAC neden doğru sıra?
2. Yeniden oynatma nasıl engellenir?
3. OAEP ve PSS hangisi ne için?
4. Ed25519 ve X25519 farkı?

## Bölüm 3–4 — cevaplar

---

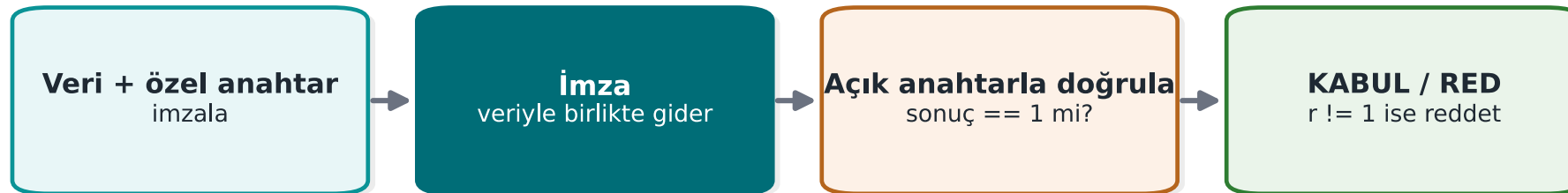
1. Alıcı **önce MAC'i** doğrular, geçmezse çözmez → değiştirilmiş metin hiç işlenmez (dolgu kâhini kapanır).
2. **Tazelik**: nonce/sayaç, zaman damgası + pencere, tek-kullanımlık challenge.
3. **OAEP** = RSA **şifreleme** dolgusu; **PSS** = RSA **imza** dolgusu.
4. **Ed25519** = imza (EdDSA); **X25519** = anahtar değişimi (ECDH). Aynı eğri ailesi, farklı iş.

## 5. Dijital imza ve tuzakları

# Dijital imza — şema

## Dijital imza: oluşturma ve doğrulama

en sık hata: if(r) ile denetlemek



API 1=başarı, 0=başarısız, <0=hata döner; if(r) hatayı da 'doğru' sayar.

# İmza · ne sağlar?

---

- **Bütünlük:** mesaj değişmedi.
- **Kimlik/inkâr edilemezlik:** gizli anahtar sahibi imzaladı.
- **Doğrulama:** açık anahtarla, herkes yapabilir.

# İmza · nerede kullanılır?

---

- Yazılım/güncelleme imzalama.
- Sertifikalar (CA imzası).
- Belge/işlem imzalama.
- TLS'te sunucu kimliği (CertificateVerify).

## Tuzak 1 · dönüş değerini denetlememek

```
int r = EVP_DigestVerify(ctx, imza, n, veri, m);  
if (r) guncellemeyi_kur(); /* HATALI */
```

Negatif hata değeri de "doğru" sayılır. Doğrusu: `r == 1`.

## Tuzak 2 · sürümü imzaya katmamak

---

- İmza yalnız içeriği kapsıyorsa, saldırgan **eski** ama geçerli imzalı sürümü yükleyebilir (downgrade).
- İmzalı içerik **sürüm numarasını** da kapsamalı; eski sürüm reddedilmeli.

## Tuzak 3 · yanlış anahtarla doğrulama

---

- Doğrulama anahtarı güvenilir mi? Nereden geldi?
- Saldırganın anahtarıyla doğrularsanız, onun imzası "geçerli" olur.
- Anahtar **sabitlenmeli** ya da güvenilir zincirden gelmeli.

## İmza · özet kuralı

---

- İmza aslında **özetin** imzalanmasıdır.
- Zayıf özet (MD5/SHA-1) → imza zayıf.
- SHA-256+ kullan.

## 6. Diffie–Hellman ve MITM

## DH · ne yapar?

---

- İki taraf, gizli anahtar **paylaşmadan** ortak bir sır türetir.
- Ağ üzerinden anahtar anlaşması.
- Modern hali: X25519.

## DH · adım adım (kavram)

Ali:  $a$  gizli,  $A = g^a$  açık gönderir  
Veli:  $b$  gizli,  $B = g^b$  açık gönderir  
Ortak sır:  $A^b = B^a = g^{(ab)}$

Dinleyen  $g^a$ ,  $g^b$  görür ama  $g^{(ab)}$  'yi hesaplayamaz.

## Kimliksiz DH · MITM riski

---

- Ali ve Veli birbirinin kim olduğunu **doğrulamıyorsa...**
- Saldırgan **araya girer**: Ali ile ayrı, Veli ile ayrı DH yapar.
- İkisini de dinler/değiştirir.

## MITM · şema

---

Ali ↔ [Saldırgan] ↔ Veli  
iki ayrı DH; saldırgan ortada

Her iki taraf "güvenli kanal kurdum" sanır.

## Çözüm · kimlikli DH

---

- DH değerleri, kimliği bilinen bir anahtarla **imzalanır**.
- TLS 1.3'te `CertificateVerify`.
- Karşı taraf bu kimliği **doğrular** (sertifika zinciri).

## İleri gizlilik (forward secrecy)

---

- Her oturum için **yeni** DH anahtarı.
- Uzun vadeli anahtar sızsa bile **eski** oturumlar çözülemez.
- Modern TLS bunu sağlar.

## Bölüm 5–6 — kısa sinama

---

1. İmza doğrulamada `if (r)` neden hatalı?
2. Sürüm imzaya neden katılmalı?
3. Kimliksiz DH'ye MITM nasıl olur? Çözüm?
4. İleri gizlilik ne sağlar?

## Bölüm 5–6 — cevaplar

---

1. Doğrulama 1=başarı, 0=başarısız, <0=hata döndürür; `if (r)` hatayı (–1) da "doğru" sayar. `r == 1` kontrol edin.
2. Aksi halde saldırgan **eski/güvensiz sürümü** aynı imzayla "geçerli" gösterir (downgrade). Sürüm imzalı veriye dahil olmalı.
3. Ortadaki her iki tarafla ayrı anahtar kurar. Çözüm: DH'yi **kimlik doğrulamayla** bağla (imzalı DH / sertifika).
4. Her oturum **geçici (ephemeral)** anahtar; uzun vadeli anahtar sızsa bile **geçmiş oturumlar** çözülemez.

## 7. Açık anahtar altyapısı (PKI)

## Sorun · kime güveneceğiz?

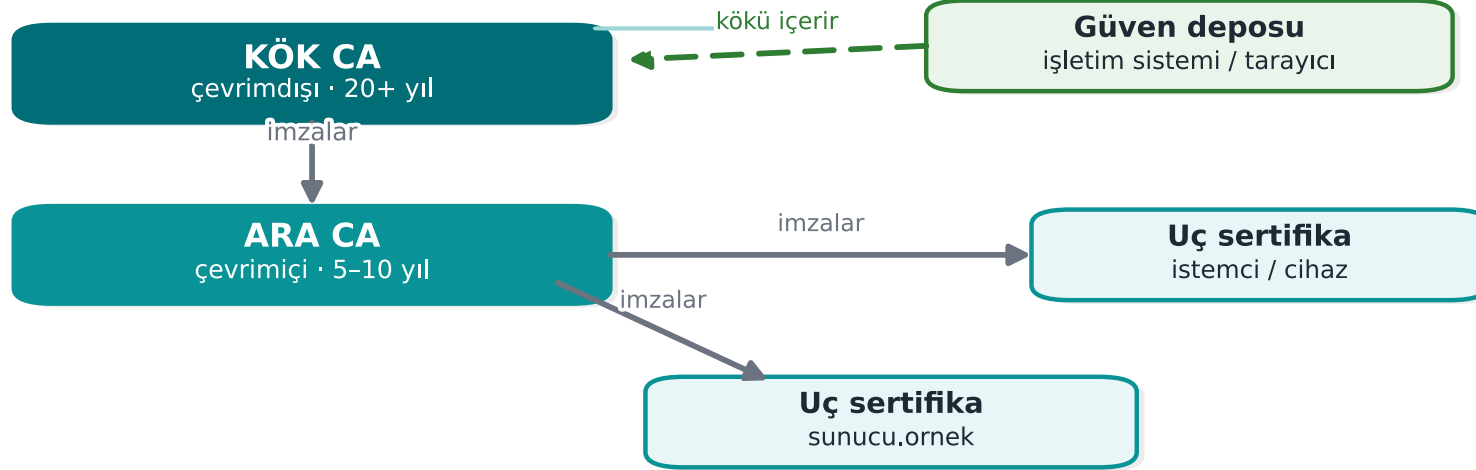
---

- Bir açık anahtar gördünüz. Gerçekten o kişiye/siteye mi ait?
- Kötü niyetli biri kendi anahtarını "banka" diye sunabilir.
- PKI bu güven sorununu çözer.

# Güven zinciri

## PKI: kim kime güvenir?

güven kökten uca akar



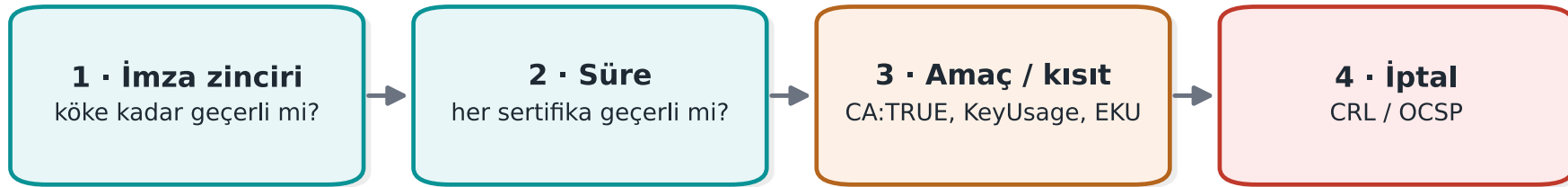
İstemci yalnız KÖKE güvenir; zincir kökten uca kesintisiz kurulamazsa doğrulama başarısızdır.

Her seviye bir alttakini **imzalar**.

# Zincir doğrulama — şema

## Zincir doğrulamanın dört sorusu

+ ad denetimi (SAN) — beşi birden geçmeden kabul edilmez



Eksik ara sertifika zinciri kurdurmaz; fail-open ile 'geçti' saymak en sık hatadır.

## Kök ve ara CA

---

- **Kök CA:** çevrimdışı, kendini imzalar, çok korunur.
- **Ara CA:** günlük sertifikaları çıkarır.
- Kök sızarsa **felaket**; bu yüzden ara CA kullanılır.

## Güven deposu (trust store)

---

- İşletim sistemi/tarayıcı, güvenilen **kök CA'ları** taşır.
- Bir zincir bu köklerden birine ulaşırsa güvenilebilir.
- Ulaşamıyorsa → güvenilemez.

## 8. X.509 sertifikası

## X.509 alanları — şema

### X.509 sertifikasında ne var?

doğrulama bu alanların HEPSİNİ denetler

**Konu (Subject) + SAN**

kime ait — ad denetimi SAN'a bakar, CN'e değil

**Açık anahtar (SPKI)**

pinning bunu sabitler

**Geçerlilik aralığı**

notBefore / notAfter

**Uzantılar**

basicConstraints (CA:TRUE), KeyUsage, EKU

**CA imzası**

zincirin bir üst halkası tarafından

Yalnız 'imza geçerli' demek yetmez: ad, süre, amaç ve iptal de denetlenir.

# İçinde ne var?

---

- **Konu (subject):** kime ait (alan adı).
- **Açık anahtar.**
- **Geçerlilik:** başlangıç/bitiş.
- **Veren (issuer):** hangi CA imzaladı.
- **İmza.**

## SAN · ad denetimi

---

- **SAN (Subject Alternative Name):** sertifikanın geçerli olduğu **alan adları**.
- Ad denetimi **CN'e değil, SAN'a** yapılır.
- `example.com` için sertifika `baska.com` 'da geçerli olmamalı.

## 9. Zincir doğrulama

## Dört soru

---

Bir sertifika zinciri doğrularken:

1. **İmza** geçerli mi? (her seviye)
2. **Zincir** güvenilir köke ulaşıyor mu?
3. **Süre** dolmamış mı?
4. **Ad** eşleşiyor mu (SAN)?

Dördü de **evet** olmalı.

## OpenSSL ile · örnek

---

```
openssl verify -CAfile kok.crt \  
-untrusted ara.crt sunucu.crt
```

- `-CAfile` : güvenilen kök.
- `-untrusted` : ara sertifika(lar).

## Sık hata · eksik ara sertifika

---

- Sunucu **ara** sertifikayı göndermezse zincir köke ulaşmaz.
- `verify` başarısız olur; `-untrusted ara.crt` ile geçer.
- Sahada çok yaygın yapılandırma hatası.

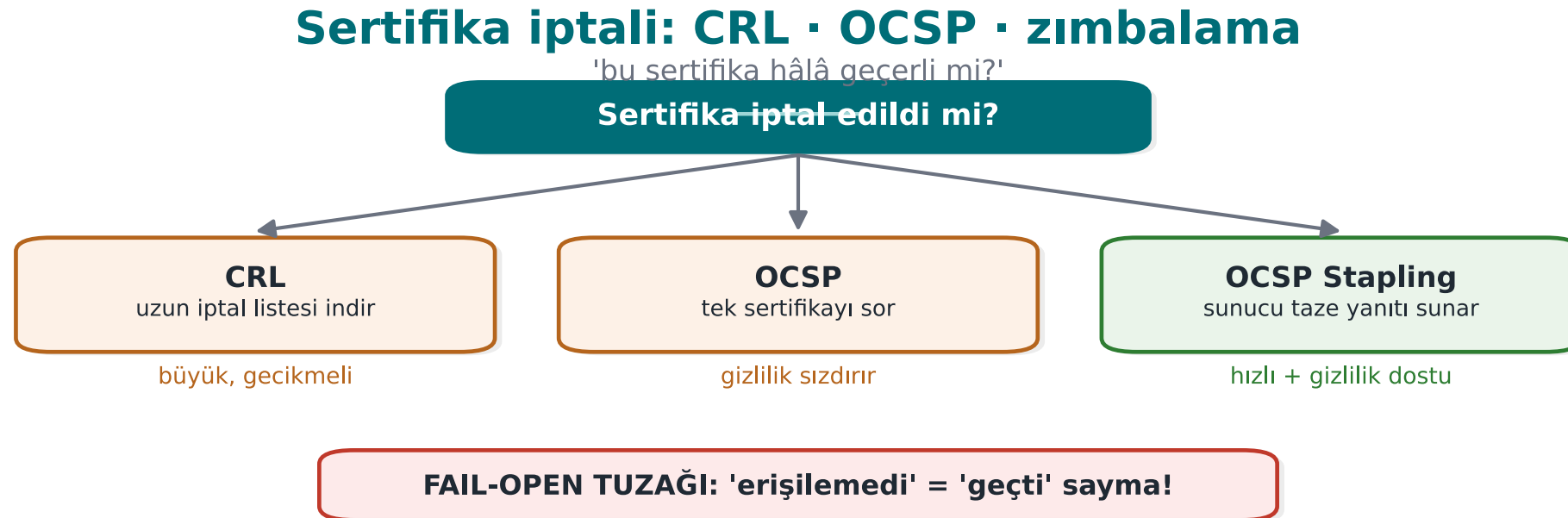
## SPKI sabitleme (pinning)

---

- Uygulama, beklenen sunucu anahtarının **özetini** gömer.
- Sahte ama "geçerli" sertifika bile kabul edilmez.
- **Yedek pin** şart (anahtar değişince kilitlenmemek için).

# 10. Sertifika iptali

# Sertifika iptali — şema



## Neden iptal?

---

- Bir sertifikanın gizli anahtarı sızarsa, süresi dolmadan **iptal** gerekir.
- İptal edilmiş sertifika kabul edilmemeli.

# CRL

---

- **CRL:** iptal edilmiş sertifikaların **periyodik listesi**.
- Büyük olabilir, güncel olmayabilir.

# OCSP

---

- **OCSP:** bir sertifikanın durumunu **anlık** sorma.
- Daha güncel; ama gizlilik/hız sorunları.

## OCSP zımbalama (stapling)

---

- Sunucu, OCSP yanıtını **kendisi** getirip sertifikayla sunar.
- Gizlilik + hız kazanır.

## Fail-open tuzağı

---

- OCSP'ye ulaşamayınca sertifikayı **kabul etmek** = fail-open.
- Saldırgan OCSP'yi engelleyip iptal edilmiş sertifikayı geçirebilir.
- Azaltma: **Must-Staple**, kısa ömürlü sertifikalar.

## Bölüm 7–10 — kısa sinama

---

1. Zincir doğrulamanın dört sorusu?
2. Ad denetimi neye yapılır (CN mi SAN mı)?
3. Eksik ara sertifika neden `verify` 'ı bozar?
4. Fail-open nedir, nasıl azaltılır?

## Bölüm 7–10 — cevaplar

---

1. (1) zincir kök'e **geçerli** mi, (2) **süre** içinde mi, (3) **amaç/kısıt** (CA:TRUE, KeyUsage/EKU), (4) **iptal** (CRL/OCSP).
2. **SAN** (Subject Alternative Name); CN artık kullanılmaz.
3. Zincir kök'e **kurulamaz**; güven yolu tamamlanmaz → başarısız. Sunucu **ara** sertifikayı da göndermeli.
4. **Fail-open**: hata/erişilemezlikte "geçti" saymak. Azaltma: **fail-closed**, OCSP stapling / must-staple, hatayı reddet.

# 11. Anahtarı donanımda saklamak

# HSM nedir?

---

- **HSM:** anahtarları saklayan/işleten özel donanım.
- Anahtar HSM'in **dışına çıkmaz**; "şunu imzala" dersiniz, sonuç döner.
- Kurcalamaya dirençli.

## PKCS#11

---

- Anahtar modülleriyle konuşmanın **standart arayüzü**.
- Uygulama anahtarın değerini **görmez**.
- HSM'ler ve yazılım benzetimleri bu arayüzü sunar.

# SoftHSM

---

- HSM'in **yazılım benzetimi**; aynı PKCS#11 arayüzü.
- Geliştirme/test için; **gerçek donanım koruması yok**.
- Üretimde gerçek HSM.

# Neden donanım?

---

- Yazılımda anahtar, beyaz kutu saldırgana açık (11. hafta).
- Donanımda anahtar yalıtılmış → çok daha güçlü.
- Mümkünse anahtarı **yazılıma hiç koyma**.

# Anahtar yaşam döngüsü

| Aşama          | Ne yapılır                 |
|----------------|----------------------------|
| Üretim         | Güçlü rastgelelik (CSPRNG) |
| Saklama        | HSM/TEE ya da korunmuş     |
| Kullanım       | En az yetki, sabit zamanlı |
| Kripto-periyot | Ömür sınırı                |
| Yenileme       | Düzenli                    |
| İmha           | Bellekten güvenli silme    |

## 12. Kuantum sonrası kriptoloji

# Tehdit

---

- Yeterince güçlü kuantum bilgisayar, bugünkü **RSA/ECC**'yi kırabilir.
- "Şimdi topla, sonra çöz" saldırısı: bugün şifreli veriyi sakla, gelecekte çöz.

## PQC · yaklaşım

---

- Kuantuma dayanıklı algoritmalar (ör. ML-KEM anahtar kapsülleme).
- Standartlaşma sürüyor.
- **Kripto-çeviklik:** algoritmayı kolay değiştirebilecek tasarım.

## Bugün ne yapmalı?

---

- Kripto-çevik ol (algoritmayı sabitleme).
- Uzun ömürlü sırlar için PQC'yi izle.
- Panik yok; ama hazırlık.

## 13. Proje: bu hafta (S8, S11)

## Proje · S8/S11

---

- [ ] **Algoritma envanteri:** amaç, algoritma, kip, anahtar uzunluğu, kütüphane.
- [ ] **Anahtar yaşam döngüsü** tablosu.
- [ ] TLS/sertifika doğrulaması; varsa sabitleme + yedek pin.
- [ ] İmza doğrulaması (güncelleme dosyasında).

# Çözümlü kendini sinama

# Soru 1

---

**RSA-2048 + AES-256 sisteminin güvenlik düzeyi?**

**Cevap:** ~112 bit (en zayıf halka RSA-2048). 128 bit için RSA-3072 ya da ECC.

## Soru 2

---

"Dolgu geçersiz" ve "MAC geçersiz" ayrı iletiler: risk?

**Cevap:** Dolgu kâhini; saldırgan yanıtlara bakıp mesajı çözebilir. Tek ve aynı hata; tercihen AEAD.

## Soru 3

---

`if (EVP_DigestVerify(...))` neden hatalı?

**Cevap:** Negatif hata değeri de doğru sayılır; denetim `== 1` olmalı. Ayrıca imzalı içerik sürümü kapsamalı.

## Soru 4

---

**verify** ara sertifika olmadan neden başarısız?

**Cevap:** Zincir köke ulaşamaz; ara sertifika eksik. Sahada sunucu ara sertifikayı göndermiyordur.

## Soru 5

---

**OCSP'ye ulaşılamayınca kabul: hangi örüntü, nasıl azaltılır?**

**Cevap:** Fail-open (yumuşak başarısızlık). Must-Staple ya da kısa ömürlü sertifikalar.

## Soru 6

---

**Kimliksiz DH'ye MITM nasıl engellenir?**

**Cevap:** DH değerleri kimliği bilinen anahtarla imzalanır (TLS 1.3 CertificateVerify) ve karşı taraf doğrular.

## Soru 7

---

**Zincir doğrulamanın dört sorusu?**

**Cevap:** İmza geçerli mi, güvenilir köke ulaşıyor mu, süre dolmamış mı, ad (SAN) eşleşiyor mu.

## Soru 8

---

**Anahtarı neden yazılıma koymamalı? Alternatif?**

**Cevap:** Beyaz kutu saldırgan yazılımdaki anahtarı çıkarabilir. Alternatif: HSM/TEE (PKCS#11); yoksa whitebox + katman (11. hafta).

## Sözlük

| Terim            | Anlam                         |
|------------------|-------------------------------|
| AEAD             | Şifreleme + bütünlük birlikte |
| Encrypt-then-MAC | Doğru sıra                    |
| OAEP/PSS         | RSA şifre/imza dolgusu        |
| Ed25519/X25519   | İmza / anahtar anlaşması      |
| SAN              | Sertifika ad denetimi         |
| CRL/OCSP         | İptal listesi / anlık sorgu   |

## Özet: bu haftanın tek cümlesi

---

Kriptoda güvenlik doğru algoritmadan çok doğru **kullanımdadır**: AEAD, doğru dolgu, doğrulanan imza, denetlenen zincir ve iyi yönetilen anahtar.

# Gelecek hafta

11. hafta — Whitebox kriptografi

Anahtar yazılımda korunuyorsa ne olur? Beyaz kutu saldırı, tablo tabanlı WBC ve sınırları.

# Ek A · OpenSSL ile zincir (adım adım)

# Amaç

---

Küçük bir zincir kuralım:

Kök CA → Ara CA → Sunucu sertifikası.

Sonra `verify` ile doğrulayalım ve tipik hatayı görelim.

## Adım 1 · kök CA (kendini imzalar)

---

```
openssl req -x509 -newkey ed25519 \  
-keyout kok.key -out kok.crt \  
-subj "/CN=Ders Kok CA" -days 3650 -nodes
```

Kök kendini imzalar; çevrimdışı tutulur.

## Adım 2 · ara CA (kök imzalar)

---

```
openssl req -newkey ed25519 -keyout ara.key \  
    -out ara.csr -subj "/CN=Ders Ara CA" -nodes  
openssl x509 -req -in ara.csr -CA kok.crt -CAkey kok.key \  
    -CAcreateserial -out ara.crt -days 1825
```

## Adım 3 · sunucu sertifikası (ara imzalar)

---

```
openssl req -newkey ed25519 -keyout sunucu.key \  
    -out sunucu.csr -subj "/CN=ornek.test" -nodes  
openssl x509 -req -in sunucu.csr -CA ara.crt -CAkey ara.key \  
    -CAcreateserial -out sunucu.crt -days 365
```

## Adım 4 · doğrula (eksik ara)

---

```
openssl verify -CAfile kok.crt sunucu.crt  
# HATA: unable to get local issuer certificate
```

Zincir köke ulaşamıyor: **ara sertifika eksik.**

## Adım 5 · doğrula (ara ile)

---

```
openssl verify -CAfile kok.crt \  
    -untrusted ara.crt sunucu.crt  
# OK
```

Ara sertifikayı verince zincir tamamlanır.

# Ders

---

- Sunucu, **ara** sertifikayı da göndermeli.
- Eksikse istemci "issuer bulunamadı" der.
- Sahadaki en yaygın TLS yapılandırma hatası.

## Zincirdeki dört soru · uygulaması

---

- **İmza:** her seviye üsttekince imzalı ✓
- **Kök:** `-CAfile kok.crt` güven deposu ✓
- **Süre:** `-days` içinde ✓
- **Ad:** uygulamada SAN denetimi (verify ad denetlemez!)

# Ek B · Kripto tasarım vakası

# Kurgu

---

Bir mobil uygulama:

- Yerel hassas veriyi şifreliyor.
- Sunucuyla güvenli konuşuyor.
- Güncellemeleri doğruluyor.

Kripto tasarımı yapalım.

## Yerel veri şifreleme

---

- **AES-256-GCM** (AEAD): gizlilik + bütünlük.
- Nonce sayaç/rastgele, **tekrarsız**.
- Anahtar TEE/HSM'de; yoksa whitebox (11).

# Sunucu iletişimi

---

- **TLS 1.3:** X25519 anahtar anlaşması + AEAD.
- Sertifika zinciri doğrulama + (varsa) SPKI sabitleme + yedek pin.
- Fail-open değil.

# Güncelleme doğrulama

---

- **Ed25519** imza; `verify == 1`.
- İmzalı içerik **sürümü** kapsar; downgrade reddi.
- Doğrulama anahtarı sabit/gömülü (bütünlükle korunur).

## Anahtar envanteri (S8)

| Anahtar         | Amaç            | Alg/uzunluk   | Saklama           |
|-----------------|-----------------|---------------|-------------------|
| Veri anahtarı   | Yerel şifreleme | AES-256       | TEE/whitebox      |
| Oturum anahtarı | TLS             | X25519 türevi | Bellek, kısa ömür |
| İmza doğrulama  | Güncelleme      | Ed25519 açık  | Gömülü            |

# Ek C · Karar tabloları

## Hangi kip?

| İhtiyaç                      | Seçim                        |
|------------------------------|------------------------------|
| Gizlilik + bütünlük          | AES-GCM / ChaCha20-Poly1305  |
| Yalnız hız (donanım AES yok) | ChaCha20-Poly1305            |
| Asla                         | ECB                          |
| Eski sistem CBC              | + encrypt-then-MAC, tek hata |

## Hangi asimetrik?

---

| İhtiyaç           | Seçim                    |
|-------------------|--------------------------|
| İmza (modern)     | Ed25519                  |
| Anahtar anlaşması | X25519                   |
| RSA şifreleme     | RSA-OAEP ( $\geq 3072$ ) |
| RSA imza          | RSA-PSS                  |

## Karar kuralı

---

- Modern, standart, iyi kullanılan seç.
- AEAD öncelikli; ECC öncelikli.
- Kararı ve gerekçesini **S8**'e yaz.

# Ek D · On sık kriptto hatası

## Hata 1 · kendi kriptonu yazmak

---

- **Yanlış:** kendi "şifreleme" algoritman.
- **Doğru:** denenmiş kütüphane, standart algoritma.

## Hata 2 · ECB kipi

---

- **Yanlış:** ECB (desen sizar).
- **Doğru:** AEAD (GCM / ChaCha20-Poly1305).

## Hata 3 · IV/nonce tekrarı

---

- **Yanlış:** sabit ya da tekrar eden nonce.
- **Doğru:** benzersiz, tekrarsız nonce.

## Hata 4 · şifreleme ama bütünlük yok

---

- **Yanlış:** yalnız CBC, MAC yok.
- **Doğru:** AEAD ya da encrypt-then-MAC.

## Hata 5 · ham RSA / yanlış dolgu

---

- **Yanlış:** ham RSA, PKCS#1 v1.5.
- **Doğru:** OAEP (şifre), PSS (imza).

## Hata 6 · zayıf özet

---

- **Yanlış:** MD5, SHA-1.
- **Doğru:** SHA-256+.

## Hata 7 · imza dönüşünü denetlememek

---

- Yanlış: `if (verify(...))`.
- Doğru: `== 1` ve sürüm kapsanır.

## Hata 8 · zincir/ad denetlememek

---

- **Yanlış:** yalnız imzaya bakmak.
- **Doğru:** dört soru + SAN denetimi.

## Hata 9 · fail-open iptal

---

- **Yanlış:** OCSP'ye ulaşamayınca kabul.
- **Doğru:** Must-Staple, kısa ömür.

## Hata 10 · anahtarı düz gömmek

---

- **Yanlış:** `static uint8_t k[16]` .
- **Doğru:** HSM/TEE; yoksa whitebox + katman (11).

# Ek E · TLS'i kodda kurmak (eleştirel)

## Sertifika doğrulamayı kapatma

```
/* ASLA: */  
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
```

Bu, MITM'e kapıyı açar. "Test için" diye bırakılan bu satır sahada felakettir.

## Ad denetimini atlamak

---

- Zincir geçerli ama **ad** doğrulanmıyorsa, saldırganın geçerli sertifikası kabul edilir.
- Ana bilgisayar adı (SAN) mutlaka denetlenir.

## Sabitleme · dikkatli

---

- SPKI sabitleme güçlüdür ama **yedek pin** olmadan anahtar değişince kilitlenirsiniz.
- En az bir yedek pin.

## Eleştirel okuma kuralı

---

Bir TLS kurulum kodunu okurken sorun:

- Doğrulama açık mı?
- Ad denetleniyor mu?
- Hata durumunda **fail-closed** mı?

# Ek F · Hızlı başvuru

## Sözlük (ek)

| Terim           | Anlam                          |
|-----------------|--------------------------------|
| Nonce           | Tek kullanımlık sayı           |
| KDF             | Anahtar türetme fonksiyonu     |
| İleri gizlilik  | Eski oturumlar sonra çözülemez |
| Trust store     | Güvenilen kök CA'lar           |
| Kripto-periyot  | Anahtar ömrü                   |
| Kripto-çeviklik | Algoritmayı kolay değiştirme   |

## Kripto kontrol listesi

---

- [ ] AEAD (GCM), nonce tekrarsız
- [ ] Bütünlük var (AEAD/EtM)
- [ ] RSA-OAEP/PSS ya da Ed25519/X25519
- [ ] İmza ==1 + sürüm kapsar
- [ ] Zincir + SAN doğrulanıyor, fail-closed
- [ ] Anahtar yaşam döngüsü + saklama belgeli

## Son söz (10. hafta)

---

Kriptoyu "açık/kapalı" değil, **doğru kullanılıp kullanılmadığıyla** değerlendirin.

Doğru kip, doğru dolgu, doğrulanan imza, denetlenen zincir, iyi yönetilen anahtar.